

Lecture 1 - Introduction **GUIDELINES**

1. Think throughout the problem

- Before approaching the solution or even before starting to think about the solution, think through the problem.
- Make sure you understand the problem.
- Make sure to clear any unclear part before designing the solution.

Approaching the solution

In software engineering we try to find a technical solution for a business problem. How can we achieve this solution in the best way possible?

- Divide and conquer
- KISS
- Learn, especially from mistakes
- Always remember why software exists
- Remember that you are not the user

Divide and Conquer

- divide the problem into smaller problems.
- Make it manageable and easily understandable.
- Try to find the perfect balance between priority and clarity.

KISS

- Keep it simple and stupid.
- Do not deliberately make your solution complex.
- Do not overthink or over engineer.

Learn from the mistakes

- Embrace the change.
- Always anticipate the changes as much as possible.
- Do not over engineer it, but keep the provisions to extend.

Reason software Exists

- Keep in mind the bigger why this software exists.
- Loss of the bigger might cause following a wrong path.

You won't be using the software

- End user is not technically capable as same as you are.
- Do not assume that user will understand.
- User friendliness and user experience matters.

Implementing the solution

SE3040 – Application Frameworks

When we are implementing the designed solution there are some guidelines to keep in mind.

YAGNI - You Ain't Gonna Need

- Don't write code that is no use in present, but you are guessing will come in handy in the future.
- Future always change. This is a waste of time.
- Write the code you need for the moment- only that.

DRY - Don't Repeat Yourself

- Always reuse code you wrote.
- Code as best as possible and keep it generalize and reusable.

Embrace abstraction

- Make sure your system functions properly without knowing the implementation details of every component part.
- User class should be able to authenticate user without knowing where to get username and password

DRITW - Don't Reinvent The Wheel

Write code that does one thing well

- Single piece of code that do one thing and that one thing really well.
- Do not try to write the magic code that does it all.

Debugging is harder than writing code

- Make it readable as possible.
- Readable code is better than compact code.

Kaizen

- Leave it better than when you found it.

PRACTICES

- Practices are norms or set of guidelines that we should follow when we are developing code.
- Unit testing
- Code quality
- Code review
- Version controlling
- Continuous integration

Unit Testing

- Small code that tests parts of main code.
- Unit can be a class, function, module, or API.
- Checks whether each unit works as expected and gives correct output.
- Helps developers change/improve code without breaking existing features.
- Makes code testable and extensible.
- Acts as a verification mechanism for developers.
- Helps detect integration issues early.

Code Quality

- Important for maintainable code.
- Code should be readable and easy to understand.
- Must follow engineering, language, and domain best practices.
- Use tools to analyze code quality regularly.
- Detect possible errors or bad design.
- Improve performance of code.
- Avoid code smells like: high complexity, large methods/classes, unclear names, duplication, too many parameters.

Code Review

- Best practice to improve code quality.
- Focus is to improve code, not criticize developer.
- Helps find better solutions and improve performance (4+ eyes principle).
- Recommended limit: < 400 LOC per review, ~500 LOC/hour, avoid long continuous reviews (>1 hour).
- Methods: peer review, lead review, pair programming.

Version Controlling

- Code must be stored in version control systems.
- Allows safe changes without fear of breaking code.
- Enables collaboration among multiple developers.
- Removes single point of failure in codebase.
- Uses branches and tags to manage versions.

Continuous Integration (CI)

- Development practice where code is frequently integrated into shared repository.
- Developers commit multiple times per day.
- Each commit is checked with automated build/tests.
- Helps detect issues early and fix quickly.

Lecture 2 - Version Controlling

Version control is the practice of tracking and managing software code changes.

- Changes are identified using a revision number.
- Each revision has its timestamp as well as the person who done the change.

Revisions can be restored, compared and merged.

- “Management of multiple revisions of the same unit of information”

Why version controlling

- Centralized source code repository.
- Easier backups
- Easy collaborative development.

Overview of changes performed to a file.

- Access control.
- Helps in conflict resolution.

BENEFITS OF VERSION CONTROLL

- Security: Protects against accidental loss or corruption.
- Clean History: Keeps track of changes and authors.

Collaboration: Enables teams to work on the same project simultaneously.

- Branching & Merging: Allows feature development without affecting the main code.
- Scalability: Works efficiently with small and large projects.

version controlling terminology

- **Repository** – The central location where project files are stored.
- **Trunk** (Master/Main Branch) – The most stable version of the project.
- **Stage** – The process of marking files for tracking changes.
- **Commit** – A snapshot of changes in the repository.
- **Merge** – Combining branches together.
- **Branch** – A copy of the code used for feature development or bug fixes.
- **Merge Conflict** – Occurs when multiple changes conflict and need manual resolution.

Git

- Distributed version control system.
- Client get a complete clone of the source code.
- In a disaster situation full source along with all history can be restored from a client.
- Free and open source.
- Multiple branches and tags.
- Feature branches, role branches (production).
- Faster comparing to other systems (works on a linux kernel and written in C).

Support multiple protocols

- HTTP, SSH

Staging area, local commits and stashing.

- **Staging area** - Mark files to be committed.
- **Local commit** - Commit code locally without pushing into the remote branch.
- **Stashing** - Keep file changes in Stash and apply them in a later.

Git

- A version control system.
- Works locally on your machine.
- Used for tracking changes and collaboration.

GitHub

- A cloud-based hosting service for Git repositories.
- Provides UI and additional features like issue tracking, CI/CD integration.

Git Alternatives

- **Fossil** – A distributed version control system with built-in bug tracking and wiki.
- **Mercurial** – A high-performance distributed version control system.
- **Subversion (SVN)** – A centralized version control system used in legacy projects.

GitHub Alternatives

- **GitLab** – Provides CI/CD tools and DevOps integration.
- **Bitbucket** – Supports Git and Mercurial, ideal for teams using Atlassian tools.

AWS CodeCommit

- A managed Git repository service for AWS users.

Azure Repos

- A cloud-based repository service for Microsoft Azure.

Staging - What is Staging?

Staging in Git is the process of preparing your changes before committing them to the repository. It acts as a middle step between modifying files and permanently saving those changes in Git.

Why is Staging Important?

- Allows you to review your changes before committing.
- Helps in grouping related changes into a single commit.
- Ensures that only selected files are committed.

How Staging Works in Git?

1. Modify files in your working directory.
- 2.Stage the files using git add (this moves the changes to the staging area).
- 3.Commit the changes to save them in the repository using git commit.

git init – Initialize a new repository.

git clone <repository_url> – Clone an existing repository.

2. Staging & Committing

- git add <file> – Stage a specific file.
- git add . – Stage all changes.
- git commit -m "Commit message" – Commit staged changes with a message

Git Commands for Staging

- git add <file> → Stage a specific file
- git add . → Stage all modified files
- git status → Check which files are staged
- git reset <file> → Unstage a file

3. Branching & Merging

- git branch <branch_name> – Create a new branch.
- git checkout <branch_name> – Switch to a branch.
- git merge <branch_name> – Merge a branch into the current branch.
- git branch -d <branch_name> – Delete a branch (after merging).

4. Working with Remote Repositories

- git remote add origin <repository_url> – Link a local repository to a remote one.
- git push origin <branch_name> – Push changes to the remote repository.
- git pull origin <branch_name> – Pull changes from the remote repository.
- git fetch – Fetch updates from the remote repository.

Undoing Changes

- git reset --soft HEAD~1 – Undo the last commit while keeping / changes.
- git reset --hard HEAD~1 – Undo the last commit and discard changes.
- git checkout -- <file> – Discard changes in a specific file

Lecture 09 - framework selection categories - **Choosing the wrong framework can lead to:** • Increased development time and costs • Technical debt accumulation • Difficulty in hiring and team scaling • Performance bottlenecks • Maintenance challenges . **A framework decision is often a long-term commitment that affects:** • Development velocity • System performance and scalability • Team productivity and satisfaction • Maintenance costs over the project lifetime • Ability to adapt to changing requirements **Key Decision Factors**

1. Project Requirements Analysis • Application Type • Performance Requirements • Scalability Considerations

2. Team Consideration • Existing Expertise • Team Size and Structure • Hiring and Talent Pool

3. Technical Considerations • Ecosystem Maturity • Community and Support • Performance Characteristics • Integration Requirements

4. Business Considerations • Time to Market • Budget Constraints • Long-term Viability ..

Frontend Frameworks - **React Ecosystem** • **Best for:** Large-scale applications, teams with React experience, rich ecosystems • **Considerations:** Requires additional libraries for routing, state management • **When to choose:** Need flexibility, large talent pool, extensive third-party support

Next.js: Full-stack React framework with SSR/SSG, excellent for SEO-critical applications. **2. Vue.js** • **Best for:** Progressive enhancement, teams wanting gentle learning curve • **Considerations** : Smaller ecosystem than React, but growing rapidly • **When to choose:** Rapid development, developer experience priority, gradual migration .

Angular • **Best for:** Enterprise applications, teams wanting opinionated structure • **Considerations** : Steeper learning curve, more verbose • **When to choose:** Large enterprise projects, TypeScript-first teams, need comprehensive solution .

Svelte/ SvelteKit • **Best for** : Performance-critical applications, small bundle size priority • **Considerations:** Smaller community, newer ecosystem • **When to choose:** Performance is paramount, team willing to adopt newer technology

5. **Solid.js** • **Best for:** React-like development with better performance • **Considerations:** Young ecosystem, smaller community • **When to choose:** Fine-grained reactivity needs, performance-first approach .

Backend Frameworks

1. Node.js Frameworks

a. Express.js • **Best for:** APIs, microservices, teams needing flexibility • **Considerations:** Minimalist, requires additional libraries for structure • **When to choose:** Small to medium projects, need maximum control

b. Nest.js • **Best for:** Enterprise applications, microservices architecture • **Considerations:** Opinionated, Angular-inspired architecture • **When to choose:** Large teams, need structure and TypeScript support

Fastify • **Best for:** High-performance APIs • **Considerations:** Similar to Express but faster • **When to choose:** Performance-critical services

2. Python Frameworks

a. Django • **Best for:** Full-stack web applications, rapid development • **Considerations:** Batteries-included, opinionated • **When to choose:** Need admin interface, ORM, built-in authentication

b. Flask • **Best for:** Microservices, APIs, smaller applications • **Considerations:** Minimalist, flexible • **When to choose:** Need flexibility, small to medium projects

c. FastAPI • **Best for:** Modern APIs, microservices, ML model serving • **Considerations:** Async support, automatic documentation • **When to choose:** Need OpenAPI docs, async operations, type hints

3. **Java/JVM Frameworks**

a. Spring Boot • **Best for:** Enterprise applications, microservices • **Considerations:** Comprehensive, enterprise-grade • **When to choose:** Large organizations, Java expertise, need mature ecosystem

b. Micronaut • **Best for:** Microservices, cloud-native applications • **Considerations:** Low memory footprint, fast startup • **When to choose:** Serverless, containerized environments

3. .NET Frameworks

a. ASP.NET Core • **Best for:** Enterprise applications, cross-platform development • **Considerations:** Microsoft ecosystem, excellent tooling • **When to choose** : .NET expertise, Azure deployment, enterprise environments

b. Ruby on Rails • **Best for:** Rapid development, startups, MVPs • **Considerations:** Convention over configuration • **When to choose** : Speed of development priority, proven patterns needed .

The Selection Process - This is the ideal way to pick a framework — a 5-step process:

1. Requirement Gathering — What does your app actually need? / **2. Initial Screening** — Eliminate options that clearly don't fit / **3. Detailed Evaluation** — Score the remaining options / **4. Proof of Concept** — Build a small prototype to test your top choice / **5. Final Decision** — Commit based on evidence .

Framework Red Flags: • Infrequent updates or abandoned development • Major security vulnerabilities not being addressed • Declining community activity • Frequent breaking changes without clear migration paths • Poor documentation or outdated tutorials • Heavy reliance on single maintainer .

Decision Making Red Flags: • Choosing based solely on popularity or hype • Ignoring team capabilities and learning curves • Overlooking long-term maintenance implications • Selecting frameworks that don't match project scale • Following trends without evaluation .

Popular Full-stack Combinations • MERN Stack • MEAN Stack • Next.js + Backend • Django + React/Vue • Spring Boot + Angular .

Migration Strategies - When you need to switch frameworks, you have 3 approaches:

Strangler Fig 🌿 — Slowly replace the old system piece by piece while keeping it running. Low risk, takes longer. // **Big Bang Rewrite** ⚡ — Throw away the old system and rewrite everything at once. Fast, but risky. Only practical for small apps. // **Hybrid** 🏗️ — Keep old framework for existing features, use new one for new features, connect them via an API layer. Can coexist long-term.

Key Takeaways - The 5 things to remember from this whole lecture :

1. There is no perfect framework — every choice has trade-offs /
2. Team knowledge beats theoretical superiority — the best framework is one your team can actually use /
3. Start simple — don't over-engineer from day one /
4. Document your decision — write down why you chose it, for future you. /
5. Plan for change — design your architecture so switching is possible later

Criteria	Weight	Framework A	Framework B	Framework C
Performance	20%	8	7	9
Team Expertise	15%	9	5	6
Community Support	15%	9	8	6
Development Speed	15%	7	9	7
Scalability	15%	8	8	9
Ecosystem	10%	9	7	6
Documentation	10%	8	9	7
Total Score		8.15	7.45	7.5